MAURIZIO LUCIANI

SERVERLESS COMPUTING

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

MAURIZIO LUCIANI



SOFTWARE & WEB DEVELOPER

SERVERLESS COMPUTING

- Fondamenti di Programmazione a Oggetti
- Introduzione a .NET e C#
- Primi passi con C#
- Gestione delle Dipendenze
- Tipi di Dato e Gestione della Memoria
- Esercitazioni Pratiche
- Introduzione alle Web API e ai Microservizi
- Architetture Avanzate e Integrazione
- Integrazione con Database
- Verso il Cloud e la Containerizzazione (anticipazione)

ABOUT ME

APRA VARGROUP

its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

SUMMARY

- Programmazione Object Oriented
- Introduzione .Net
- Installazione ambiente di sviluppo
- Hello world in .Net 8
- Linguaggio C#
- Controller
- Accesso ai dati



Programmazione Object Oriented (OOP)

La programmazione orientata agli oggetti consiste nel raggruppare, all'interno di unità di codice chiamate **classi**, sia le strutture dati sia le procedure (metodi) che operano su di esse.
Una classe rappresenta quindi un modello astratto: durante l'esecuzione del programma, a partire da essa vengono creati (istanziati) gli **oggetti**, ciascuno dei quali è un'istanza concreta di quella classe.

Un linguaggio di programmazione si definisce orientato agli oggetti quando la sua sintassi nativa consente di implementare:

- **incapsulamento**;
- **ereditarietà**;
- **polimorfismo**.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Classi

Le classi definiscono dei tipi di dato e permettono la creazione degli oggetti secondo le caratteristiche definite nella classe stessa.

La classe è composta da:

- *Attributi,* ossia variabili e/o costanti che definiscono le proprietà degli oggetti istanziabili invocando la classe; i valori inizializzati degli attributi sono ottenuti attraverso il cosiddetto costruttore della classe;
- *Metodi*, ossia procedure che operano sugli attributi.

Oggetti

Un oggetto è una istanza di una classe. Esso è dotato di tutti gli attributi e i metodi definiti dalla classe.

Dal punto di vista del calcolatore, istanziare un oggetto vuol dire allocare memoria ed eventualmente inizializzarla secondo le specifiche definite dalla classe. Molti linguaggi forniscono un supporto per l'inizializzazione automatica di un oggetto, con uno o più metodi speciali, detti costruttori.

Il codice eseguibile del programma accede a tale zona di memoria sempre e solo secondo le modalità definite dalla classe.

Incapsulamento

Per alterare lo stato interno dell'oggetto, è necessario invocarne i metodi pubblici, ed è questo lo scopo principale dell'incapsulamento. Infatti, se gestito opportunamente, esso permette di vedere l'oggetto come una *black-box*, cioè una "scatola nera" con la quale l'interazione avviene solo e solamente tramite i metodi definiti dall'interfaccia. Il punto è dare delle funzionalità agli utenti nascondendo i dettagli legati alla loro implementazione.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Ereditarietà

Il meccanismo dell'ereditarietà permette di derivare nuove classi a partire da quelle già definite realizzando una gerarchia di classi. Una classe derivata attraverso l'ereditarietà (sottoclasse o classe figlia) mantiene i metodi e gli attributi delle classi da cui deriva (classi base, superclassi o classi madre); inoltre, può definire i propri metodi o attributi, e ridefinire il codice di alcuni dei metodi ereditati tramite un meccanismo chiamato overriding.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Polimorfismo

Nella programmazione ad oggetti, con il nome di polimorfismo si indica il fatto che lo stesso codice eseguibile può essere utilizzato con istanze di classi diverse, aventi una *superclasse* comune.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

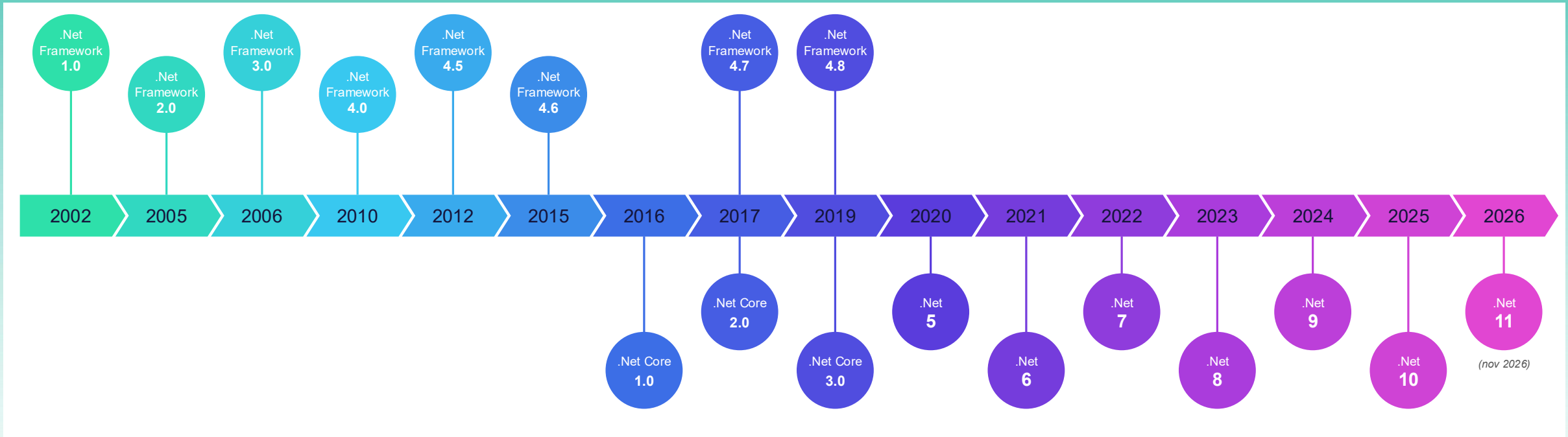
Microsoft .NET

Microsoft .NET è una piattaforma di sviluppo ideata e sviluppata da **Microsoft** che mette a disposizione varie funzionalità come **il supporto per più linguaggi di programmazione** (coprendo tutti i paradigmi di programmazione: object oriented, procedurali, funzionali, imperativi, user oriented, ecc.), modelli di programmazione asincroni e simultanei, interoperabilità, consentendo l'esecuzione su più piattaforme ed in presenza di scenari applicativi variegati.

Wikipedia

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

.NET history



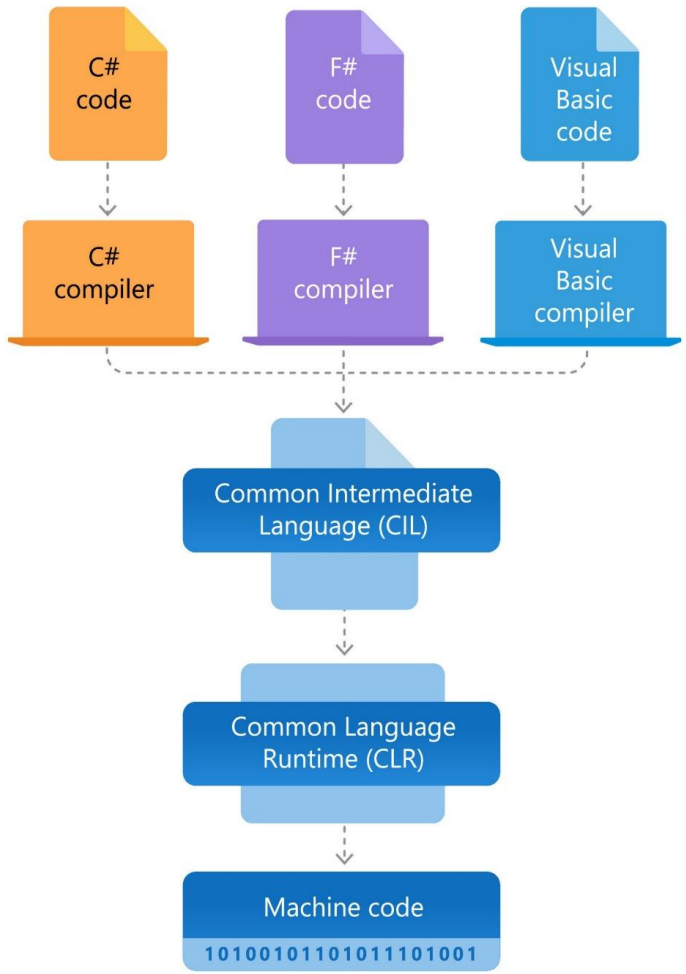
COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Linguaggio C#

- Prima versione commerciale rilasciata da Microsoft nel 2002 assieme alla prima versione di .NET
- Linguaggio derivato dal linguaggio C (linguaggio compilato)
- Linguaggio fortemente tipizzato
- Ultima versione: 14 (Giugno 2025)
- Sviluppo desktop, mobile e web
- Sviluppo di applicazioni aziendali, web, giochi, app mobile, ecc.
- Linguaggio prima compilato e poi interpretato
- IDE
  - Visual Studio (community o enterprise)
  - Visual Studio Code
  - Altri (Rider, ecc.)

Dal codice all'esecuzione

1. Scrittura del codice sorgente in linguaggio C# (o altri linguaggi .NET);
2. Compilazione tramite compilatore C#, che produce un codice intermedio (CIL - Common Intermediate Language), non ancora eseguibile, ma indipendente dal linguaggio, ovvero ottenibile anche da sorgenti scritti in altri linguaggi di programmazione;
3. Tramite il CLR – Common Language Runtime, il compilatore di .NET traduce il linguaggio intermedio in linguaggio nativo eseguibile (linguaggio macchina).



Assembly

Un programma C# che è stato compilato nel formato intermedio del CIL viene chiamato Assembly.

Un Assembly è una raccolta di tipi creati per essere usati insieme e per formare un'unità logica di funzionalità.

Gli Assembly hanno la forma di file eseguibili (*.exe*) o libreria a collegamento dinamico (*.dll*) e sono i blocchi predefiniti delle applicazioni .NET.

Gli Assembly forniscono al Common Language Runtime le informazioni necessarie per riconoscere le implementazioni dei tipi.

Tipologie:

- Process Assembly: programma completo ed autonomo;
- Library Assembly: contiene codice che può essere utlizzato da altri programmi.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE TURISMO E NUOVE TECNOLOGIE

Struttura di un programma C#

```
using System;

//NAMESPACE
namespace MyNamespace {

    //CLASS
    class MyClass {

        //METHOD
        static void MyMethod(string variable) {

            //STATEMENT
            //keyword   identifier   operator   literal   separator
            //int       i            =          0         ;
            int i = 0;

            while(i < 15)
            //CODE BLOCK
            {
                Console.WriteLine(variable);
                i++;
            }
        }
    }
}
```

- Keyword: parola chiave predefinita nel linguaggio, che ha un particolare significato per il compilatore;
- Identifier: nome che attribuiamo a variabili, metodi, classi, namespace per identificarli;
- Operator: simbolo che indica al compilatore di svolgere una certa operazione;
- Literal: valori dei tipi conosciuti dal linguaggio, come numeri o stringhe;
- Separator: caratteri che segnalano l'inizio o la fine di parti di codice;
- Statement: istruzione, secondo le regole di sintassi del linguaggio, che indica al compilatore di eseguire un compito preciso;

Struttura di un programma C#

```
using System;

//NAMESPACE
namespace MyNamespace {

    //CLASS
    class MyClass {

        //METHOD
        static void MyMethod(string variable) {

            //STATEMENT
            //keyword   identifier   operator   literal   separator
            //int       i            =          0         ;
            int i = 0;

            while(i < 15)
            //CODE BLOCK
            {
                Console.WriteLine(variable);
                i++;
            }
        }
    }
}
```

- Code Block: blocco di codice, cioè collezione di statement, tipicamente definito all'interno di parentesi graffe;
- Method: contenitore di blocchi di codice, ai quali viene assegnato un identificatore, in modo da poter essere richiamato in altre parti del programma. Può prendere in input dei parametri e può restituire un valore di ritorno.
- Class: è un tipo in C# che può contenere variabili e metodi al suo interno;
- Namespace: contenitori di classi e altri tipi (come strutture, enumerazioni, interfacce);
- Program: file sorgenti che contengono namespace. I programmi compilati vengono chiamati Assembly.

Prerequisiti

- Installazione .NET SDK (versione .NET 10)
  https://dotnet.microsoft.com/it-it/download
- Visual Studio Code
  https://code.visualstudio.com/download
- Estensione C#
  https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csharp

Dotnet CLI

.NET SDK include un’interfaccia a riga di comando molto utile in Visual Studio Code.

La Dotnet CLI permette di:

- Creare un nuovo progetto
- Creare una nuova soluzione
- Gestire le dipendenze
- Avviare un’applicazione
- Pubblicare un’applicazione

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Dotnet CLI – Comandi principali

- Creare un nuovo progetto .NET 10
  - `dotnet new`
- Compilare un progetto (viene creato l'assembly: nomeProgetto.dll)
  - `dotnet build`
- Avviare un progetto (viene prima eseguita la compilazione)
  - `dotnet run`
- Avviare un progetto con hot reload
  - `dotnet watch`
- Aggiungere una dipendenza
  - `dotnet add package nomeDipendenza`
- Ripristinare le dipendenze
  - `dotnet restore`

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione1 - Hello World

Console Application

Un'applicazione **console** o **programma a riga di comando** è un programma progettato per essere utilizzato tramite un'interfaccia utente solo testo.

- Su Visual Studio Code aprire un nuovo terminale dal menu Terminal - New Terminal (CTRL+ò)
- Spostarsi nella directory in cui si vuole creare il nuovo progetto, ad esempio per spostarsi nella cartella CorsoITS:
  - `cd <cartella_dei_progetti_its> + Invio`
  - `cd CorsoITS + Invio`
- Creare un nuovo progetto .NET 10 di tipo console denominato Esercitazione1
  - `dotnet new console -n Esercitazione1 -f net10.0`

Esercitazione1 - Hello World

Il terminale avvisa della creazione del progetto.

Aprire il progetto dal menù File/Open Folder.

All’interno della cartella del progetto sono presenti:

- bin: directory che conterrà i file binari, che costituiscono il codice eseguibile effettivo dell'applicazione, quindi gli assembly (alla creazione del progetto non contiene files);
- obj: directory che contiene gli output di compilazione intermedi creati durante il processo di compilazione;
- Esercitazione1.csproj: è il file di progetto C#;
- Esercitazione1.sln: è il file della soluzione. Una soluzione è una struttura per organizzare i progetti in Visual Studio;
- Program.cs: è il file che contiene il codice di avvio del progetto.

Esercitazione1 - Hello World

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework
>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

Esercitazione1.csproj
Il file csproj è il file di progetto, è un file XML contenente tutte le informazioni e le istruzioni richieste da MSBuild (Microsoft Build Engine) per compilare il progetto, ad esempio il contenuto da includere, i requisiti della piattaforma, le informazioni sul controllo delle versioni, le impostazioni del server Web o del server di database e le attività da eseguire.

Esercitazione1 - Hello World

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

Program.cs
Il file Program.cs è il file che contiene il codice di avvio dell'applicazione.

Alla creazione del progetto contiene un'istruzione che, se eseguita, stampa a console la scritta 'Hello, World!'.

In questo esempio, viene invocato il metodo WriteLine della classe Console.
La classe Console è contenuta in una libreria predefinita di .NET.

Fino a C# 8 il metodo Main rappresentava il punto d'ingresso .

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione1 - Hello World

Nel terminale, posizionandosi nella cartella del progetto Esercitazione1, digitare:

- `dotnet run`

Il comando esegue il codice sorgente, dopo aver eseguito la compilazione.

Nel terminale viene visualizzata la stringa ‘Hello, World!’

Nella cartella bin\Debug\net8.0 sono ora presenti dei files di output, tra cui Esercitazione1.dll, cioè l’assembly dell’applicazione.

I files temporanei vengono inseriti nella directory obj.

Nel terminale spostarsi dentro la cartella Esercitazione1\bin\Debug\net8.0 e digitare:

- `dotnet Esercitazione1.dll`

Nel terminale viene visualizzata la stringa ‘Hello, World!’

Esercitazione2 - Hello World, metodo Main

- Su Visual Studio Code aprire un nuovo terminale dal menu Terminal - New Terminal
- Nel terminale, spostarsi nella directory in cui si vuole creare il nuovo progetto, ad esempio, dalla radice, per spostarsi nella cartella CorsoITS del Desktop;
- Creare un nuovo progetto .NET 10 di tipo console denominato Esercitazione2
  - `dotnet new console -n Esercitazione2 **--use-program-main** -f net10.0`

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione2 - Hello World, metodo Main

```
namespace Esercitazione2;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello, World!");
    }
}
```

Nel namespace Esercitazione2, all'interno della classe Program, è presente il metodo statico Main, che invoca il metodo WriteLine della classe Console.
La classe Console è contenuta in una libreria predefinita di .NET, del namespace System.

A partire da C# 9 il codice è stato semplificato (Esercitazione1).

Esercitazione2Plus - Hello World, Main Args

```
class Program
{
    0 references
    static void Main(string[] args)
    {
        Console.WriteLine("Hello, World!");
        if(args.Length > 0)
        /* Controllo se ci sono argomenti da linea di comando */
        {
            Console.WriteLine("Arguments provided:");
            foreach (var arg in args)
            {
                Console.WriteLine(arg);
            }
            // TODO Restituire il 2' argomento
        }
        else
        {
            Console.WriteLine("No arguments provided.");
        }
    }
}
```

Il parametro string[] args del metodo Main permette di ricevere argomenti da riga di comando:

tutto ciò che viene scritto dopo -- nel comando dotnet run viene passato al programma come input, utile per personalizzare l'esecuzione.

Esercitazione3 - Dependency

- Nel terminale di Visual Studio Code spostarsi nella directory in cui si vuole creare il nuovo progetto;
- Creare un nuovo progetto .NET 8 di tipo console denominato Esercitazione3
  - `dotnet new console -n Esercitazione3`
- Aggiungere al progetto Esercitazione3 una dipendenza (es. Humanizer)
  - `dotnet add package Humanizer`
- Che cos'è Humanizer?

Esercitazione3 - Dependency

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Humanizer" Version="2.14.1" />
  </ItemGroup>

</Project>
```

Esercitazione3.csproj
Nel file csproj è presente un nuovo tag xml (<ItemGroup>), che contiene il riferimento al package 'Humanizer' e la sua versione.
I riferimenti ai pacchetti, tramite <PackageReference>, specificano le dipendenze dei pacchetti direttamente all'interno dei file di progetto.
All'interno della cartella bin sono presenti le dll relative a Humanizer.

Esercitazione3 - Dependency

Humanizer è un package per la manipolazione e la visualizzazione di stringhe, enum, date, orari, intervalli di tempo, numeri e quantità.

Firma del metodo Pluralize:

string string.Pluralize([bool inputIsKnownToBeSingular = true])

E’ un metodo che restituisce il plurale della stringa su cui è applicato.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione3 - Dependency

```
using Humanizer;

Console.WriteLine("Woman".Pluralize());
```

Program.cs

- Modificare il Program.cs e inserire il codice come in figura sopra che richiama il metodo Pluralize() e stampa a console il risultato restituito dal metodo;
- Eseguire l'applicazione.

A console viene visualizzata la scritta: *Women*